

PROJECT OVERVIEW

Ricoz Application Performance Monitoring & Observability Platform

Project Topic

Enterprise Application Performance Monitoring (APM) and Observability Platform. An enterprise-level software platform that provides end-to-end visibility into application health, response times, errors, and user experience.

1. Client Requirements

The client wants a platform that monitors applications and provides useful information about their performance and health.

Feature	Requirement
Application Performance Monitoring	Monitor response times, performance metrics, transactions, and traces.
Real User Monitoring (RUM)	Track real users' experiences, including page performance, loading times, and user-facing issues.
Synthetic Monitoring	Run automated checks to verify applications and important endpoints are accessible and working.
Error Detection and Diagnostics	Identify application errors, monitor error rates, and help investigate causes of failures.
Alert Management	Configure alert rules, evaluate conditions, and send notifications via email, webhooks, or PagerDuty.
Metrics and Dashboard Statistics	Store monitoring data and display time-based statistics, trends, and charts.

2. Purpose of the Project

- Detect, diagnose, and help prevent application performance problems before they seriously affect users or business operations.
- Monitor application health continuously.
- Identify slow response times and performance bottlenecks.
- Detect errors and unexpected failures.
- Notify developers when configured conditions are triggered.
- Track application availability and user experience.
- Provide historical data and dashboards for performance analysis.

3. Example: How It Works

Step	Description
1. User visits the website	A customer opens the website and tries to place an order.
2. Platform monitors performance	It collects response time, errors, and transaction details.
3. A problem is detected	The checkout API takes 5 seconds instead of its normal 300 milliseconds.
4. Alert is triggered	If the configured condition is met, the platform notifies the responsible team.
5. Developer investigates	The team uses metrics and traces to investigate and improve the checkout process.

4. How the Project Is Useful

User	Benefit
Developers	Find slow code, errors, and performance bottlenecks.
DevOps engineers	Monitor application availability and operational health.
QA engineers	Check reliability and identify performance regressions.
IT operations teams	Detect incidents and respond to alerts.
Businesses	Reduce downtime and improve customer experience.
Product teams	Understand how performance affects users.

Real-world Benefits

- Reduced downtime through earlier problem detection.
- Faster troubleshooting using metrics and traces.
- Better user experience by identifying slow pages and failing transactions.
- Improved reliability through continuous monitoring.
- Data-driven performance analysis using historical metrics.

5. Backend Architecture and Responsibilities

The backend handles monitoring data, manages alert rules, processes statistics, and supports APIs used by the frontend.

Layer	Responsibilities
Frontend Dashboard	Charts, metrics, alert rules, and application status.
Backend API (Python)	Alerts API, Source Maps API, Stats Endpoint, and Alert Evaluator.
Data Storage	PostgreSQL + TimescaleDB for time-series metrics (planned migration from SQLite, based on the shared scope).

Key Backend Responsibilities

- Store and retrieve monitoring data.
- Create, update, and manage alert rules.
- Evaluate alert conditions on a schedule.
- Manage notification channels.
- Aggregate time-series data for dashboard charts.
- Manage source map uploads and storage.

6. Difference from Basic Monitoring

Basic monitoring: “The server is running.”

APM: “The server is running, but the checkout API is slow because a database query is taking too long.”

7. Short Project Summary

Project Topic: Enterprise Application Performance Monitoring and Observability Platform.

Client Requirement: Provide end-to-end visibility into application health, response times, errors, user experience, and availability, with dashboards and alert management.

Purpose: Help development and operations teams detect, investigate, and resolve application performance issues.

Usefulness: Reduce downtime, improve reliability, speed up troubleshooting, and improve user experience.

Backend Work: Develop APIs for alert management, monitoring statistics, source maps, data storage, and scheduled alert evaluation.

In one sentence: A centralized platform that helps teams monitor applications, detect problems, and understand performance issues before they significantly affect users.